

1 从 Instrumentation 到 Trace Reading

这一节把前一节的概念落到证据层：先让 agent 产生可观察的 trace，再在 Phoenix 中读懂每一次运行的输入、输出、工具调用、模型调用、token count、cost 与 latency，最后把肉眼看到的失败整理成后续 eval 可以自动追踪的分类。Laurie Voss 在这里给出的核心判断很直接：写 eval 之前，团队必须先看 trace；否则 eval 只是在猜系统哪里会坏。

1.1 Eval 结果的解剖：score、label、explanation

一个 eval 结果通常有三个部件：数值分数 **score**、人类可读标签 **label**，以及在 LLM 裁判场景下生成的 **explanation**。可以把它理解成一个很小的 JSON 对象：**score** 像考试分数，**label** 像“通过/失败”的判定，**explanation** 像阅卷老师写在旁边的批注。批注的价值在于它给出下一步修复方向。

定义：eval 结果三元组

若把一个 LLM-as-a-judge 记为 J_θ ，它读取输入 x_i 、agent 输出 y_i 、可用上下文 c_i 与评分规则 r ，一个典型结果可以写成：

$$J_\theta(x_i, y_i, c_i; r) = (s_i, \ell_i, e_i)$$

其中 s_i 是 **score**，通常是 0、1 或一个归一化分数； ℓ_i 是 **label**，例如 **pass**、**fail**、**actionable**； e_i 是 **explanation**，说明裁判为什么给出这个判断。代码型 eval 可以返回 **score** 和 **label**，若开发者没有显式写解释逻辑，它通常没有自然语言解释。

FIGURE_SLOT: eval-result-score-label-explanation

建议插入 eval result object 截图或重绘图，突出 **score**、**label**、**explanation** 三个字段。

图 1: eval 结果对象的基本结构。¹

视频里的 Tokyo budget travel 例子说明了 **explanation** 的实际用途。用户要的是预算旅行建议，模型确实给了东京旅行推荐，可是没有说明每项建议大概花多少钱。若只问“有没有推荐东京旅行”，答案会偏向通过；若按“预算旅行”这个真实需求判断，它就漏掉了成本维度。LLM judge 的 **explanation** 把这种细微缺口暴露出来：问题并非“没有回答”，问题是回答缺少预算信息，因此 prompt 需要明确要求价格、预算档位或成本对比。

误区：只看最终答案会误判失败形态

最终答案像一张体检报告的首页，只能告诉读者“看起来大概如何”。trace 像完整病历，能显示模型看过什么资料、调用了哪些工具、每一步传递了什么上下文。agent 系统的失败常常发生在中间步骤；只看最终答案会把工具选择错误、文件写入失败、搜索证据不足和推理跳步混在一起。

当 eval 跑在成百上千条 trace 上，**explanation** 还能形成模式。某一次失败可能只是非确定性系统的偶发波动；同类解释反复出现，就更像 prompt、工具权限、数据源或需求定义上的系统性缺

¹ 视频画面时间区间：00:18:45–00:19:02。

陷。这里也有新的工作量：人要读一千条解释会很痛苦，所以后续经常再用一个 LLM 把解释聚集成失败类别。这个做法有用，同时会引入“裁判的裁判也需要验证”的下一层问题。

1.2 迭代闭环：从 Instrumentation 开始

本节的工作流可以压缩成一个循环：Instrumentation 捕获 trace，Phoenix 展示 trace 和 span，eval 读取这些证据并生成 annotation，团队分析失败，然后修改 prompt 或应用逻辑，再重新运行。这个循环的顺序很关键，因为 eval 的质量取决于它测试的证据是否真实。

FIGURE_SLOT: eval-iteration-loop

建议插入或重绘循环图：instrument → trace → evaluate → annotate → analyze → improve → repeat。

图 2: 从 Instrumentation 到改进 agent 的迭代闭环。²

核心流程

一个可操作的 eval 系统至少要保留五类对象：运行证据 trace/span、被测输入、agent 输出、eval 生成的 annotation、以及修复后的新版本结果。缺少 trace，团队只能评价表面文本；缺少 annotation，失败无法在 UI、数据集和实验中被筛选；缺少重复运行，修复是否有效只能靠感觉。

Phoenix 在这里承担的是可观察性和 eval 结果管理。它捕获 LLM call、tool call、agent step 的输入输出，也存储 eval 结果，并在 UI 中显示 annotation、attribute、模型名、token count、cost、latency、原始输入和原始输出。Phoenix Cloud 是本 workshop 使用的托管版本，优点是省掉本地安装；Phoenix 本身也可以本地运行。当前文档只复述 workshop 中的产品边界，若要写当前产品选型或采购建议，还需要另行核对官方文档。

1.3 Phoenix 设置：OpenTelemetry、OTel 与 OpenInference

实际设置的关键代码很短。讲者展示的核心形状是从 phoenix.otel 导入 register，给项目命名，并开启自动 instrumentation。

```
1 from phoenix.otel import register
2
3 tracer_provider = register(
4     project_name="ai-agent-financial-analysis",
5     auto_instrument=True,
6 )
```

Listing 1: Phoenix 注册的核心形状（根据讲解重建，待 notebook 画面核验）

FIGURE_SLOT: phoenix-register-notebook-cell

建议插入 notebook 安装与 register(..., auto_instrument=True) 单元格截图，必要时裁剪到代码和警告区域。

图 3: Phoenix 注册与自动 instrumentation 的 notebook 单元格。³

²视频画面时间区间：00:21:20–00:21:41。

背景：OpenTelemetry、OTel、OpenInference

OpenTelemetry, 常缩写为 OTel, 是通用可观察性标准; 很多服务端日志、Kubernetes 相关监控和分布式追踪都可以围绕它工作。OpenInference 是面向 LLM 应用的扩展层, 它把 prompt、completion、tool call、model、token count、cost 等 AI 特有字段放进 trace/span 的 attribute 中。Phoenix 的自动 instrumentation 正是利用这些标准, 让开发者少改业务代码就能把 agent 行为送进可观察性系统。

视频中还有一个生产环境提醒: demo 使用的 span processor 适合小规模演示; 生产环境一次可能产生大量 spans, 更常见的选择是 batch processor。类比一下, demo 里的处理器像“每收到一封信就跑一趟邮局”, 适合课堂; 生产系统更像“定时成批投递”, 否则吞吐和网络开销会很难看。

1.4 金融分析 Demo Agent: 两个 turn 与一个 root span

演示 agent 的任务故意简单: 输入股票 ticker 或一组 tickers, 第一步研究公司与市场信息, 第二步把研究结果写成简短金融报告。它使用 Claude Agent SDK, 因为这个 SDK 足够轻, 能保留对话上下文, 能调用工具, 能使用 Web Search。讲者选择 Claude Haiku, 理由是便宜、快、能力有限, 容易在 workshop 中暴露可观察的失败; 如果模型太强, 课堂上可能看不到足够多的错误样本。

金融示例的边界

本节所有 Tesla、Apple、Microsoft、Nvidia、Rivian、Coca-Cola 等案例都只是 eval 工程示例, 不构成投资建议。读者应该关注 agent 如何搜索、写作、失败和被评测, 避免把示例输出当成真实金融判断。

这个 agent 的最小机制如下: 研究 turn 用 Web Search 找资料, 写作 turn 继承前一轮上下文并生成报告。由于两个 turn 默认可能在 Phoenix 中显示成两个独立 trace, 讲者用 OpenTelemetry 包了一层 financial_report span, 让它们在一个 root span 下呈现。这样做的目的很实际: 后面要对“整次金融报告任务”打分时, 团队需要一个 parent span 或 root span 代表完整运行。

```
1 async def financial_report(ticker: str, focus: str) -> str:
2     with tracer.start_as_current_span("financial_report"):
3         research = await client.query(
4             f"Research_{ticker};_{focus}_on_{focus}.",
5             allowed_tools=["WebSearch"],
6         )
7         report = await client.query(
8             "Write_a_concise_financial_report_using_the_research_context.",
9             context=research,
10        )
11        return report
```

Listing 2: 金融报告 agent 的简化结构 (根据讲解重建, 待 notebook 画面核验)

³视频画面时间区间: 00:22:56-00:24:30。

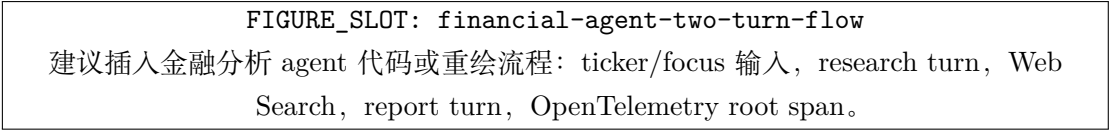


图 4: 金融分析 demo agent 的两个 turn 与一个顶层 trace。⁴

Tesla 首次运行展示了 agent 的真实动态性。它会发起多轮 Web Search，例如金融表现、2026 增长展望、季度收入与利润率、Cybertruck 产量与 Roadster 需求等；随后生成 executive summary、Q4 25 highlights、2026 financial outlook、key growth drivers、risk assessment 之类的段落。表面上看，这份报告“很像样”。可“像样”无法作为上线标准，因为 agent 每次可能搜索不同资料、调用不同工具、生成不同推理路径。

为什么 agent 不能只按输入输出写死测试

传统函数常常有稳定映射：输入 x 给出输出 $f(x)$ 。agent 更像一条可变路径：

$$x \rightarrow a_1 \rightarrow o_1 \rightarrow a_2 \rightarrow o_2 \rightarrow \cdots \rightarrow y$$

其中 a_k 是第 k 次 action，例如 Web Search、工具调用或写作 turn； o_k 是该 action 的观察结果。eval 既要看最终输出 y ，也要看中间路径是否合理。trace reading 的价值就在这里。

1.5 Phoenix UI 中读 trace：从 span 树到 attributes

Phoenix UI 把一次运行拆成可点击的 span 行。顶层行显示完整 agent span 的输入输出；展开以后，可以看到研究 turn、写作 turn、Web Search 调用、工具结果、模型调用，以及每个节点的 attribute。对这个 workshop 来说，最重要的观察是：trace 把“agent 做了什么”从黑盒文本变成可检查的执行树。

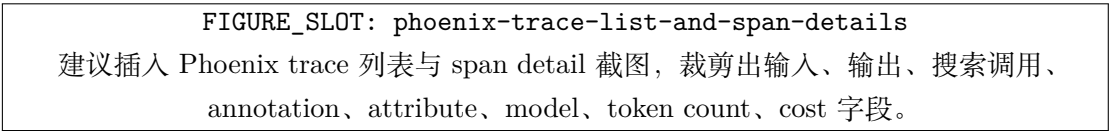


图 5: Phoenix 中的 trace 列表、span 细节与属性字段。⁵

直觉类比：trace 像监控录像，annotation 像质检标签

日志告诉工程师“系统说了什么”；trace 更接近“系统做了什么”。annotation 则是贴在 trace 或 span 上的质检标签，例如某次运行通过、失败、缺少推荐、可能幻觉。后续 Phoenix 过滤、数据集构造和实验对比，都依赖这些标签把原始运行变成可管理的证据库。

span 里的 JSON 会很长，Phoenix 的价值在于把它变成可读界面。读者应该重点看这些问题：输入是否被正确理解；工具调用是否合理；搜索 query 是否覆盖需求；中间结果是否被传给写作 turn；模型输出是否包含来源证据；token count、cost 与 latency 是否异常。一个金融报告如果最后失败，原因可能是研究资料错、工具失败、上下文丢失、报告写作指令含糊，或者输出格式偏离预期。trace 让这些原因可区分。

⁴视频画面时间区间：00:26:58–00:31:08。

⁵视频画面时间区间：00:33:24–00:35:27。

1.6 构造 13 条 trace：小型语料库的意义

单次 Tesla 运行只是一条样本，无法说明 agent 在不同任务形态下是否稳定。讲者因此在 notebook 中准备了 12 条额外测试 query，加上原始 Tesla 查询，一共 13 次运行。它们覆盖单 ticker、不同 focus area、Apple/Microsoft 双公司比较，以及不同类型公司的分析请求。这个数量对生产系统远远不够，课堂上足以展示“失败模式会开始成形”。

FIGURE_SLOT: multi-query-trace-corpus

建议插入 12 条测试 query 表或 Phoenix 中 13 条 trace 堆叠的界面截图。

图 6: 从单次 demo 扩展到 13 次运行的小型 trace corpus。⁶

这组 query 的设计思路比具体 ticker 更重要。真实用户不会总是说“research Tesla financial performance”。有人会问“What’s going on with Tesla stock?”，有人会问“Yo, is Tesla a buy right now?”，意图相近，语言形态差异很大。还要覆盖多部分问题、比较问题、冷门公司、错误 ticker、jailbreak 尝试和 adversarial 输入。生产 trace 是最好的 eval 数据来源；若新产品还没有真实用户，synthetic data 可以作为启动方案，用 LLM 生成覆盖不同意图、语气和边界条件的查询，再让 agent 产生真实 trace。

未知风险：小语料库容易制造虚假的安全感

13 条 trace 只能让课堂看到失败类型，不能代表生产分布。生产里的失败常常藏在长尾：罕见 ticker、口语化请求、多语言输入、监管敏感问题、恶意提示、工具权限边界。小样本可以启动 eval 设计，不能直接支持上线信心。

1.7 手工读 trace：先定义“好”，再分类“坏”

在写 eval 之前，团队要先定义成功标准。对这个金融分析 agent，讲者把“好报告”压成一个具体要求：报告需要给出 actionable recommendations，也就是能告诉读者 buy/sell/hold 或至少给出明确下一步；它还要区分 forward-looking analysis 与 historical summary。只说“报告要 thorough”或“要 in-depth”太空，LLM judge 和人类标注者都会各按各的理解打分。

需求先于 eval

eval 源于需求，再被写成可执行测试脚本；它是需求的可执行版本。若需求写成“报告要好”，eval 会变成主观猜测；若需求写成“必须包含面向投资决策的 buy/sell/hold recommendation，并说明关键风险与前瞻证据”，eval 才有可校准的判断边界。

这里必须引入 domain expert、product manager 和真实用户。工程师可能觉得一份报告“看起来很专业”，用户可能会直接说它没用，因为它没有回答“该不该买”。这类差异属于产品质量定义问题，不能被当作语气问题处理。trace reading 阶段越早暴露这种差异，后续 rubric 和 eval 越少返工。

Apple 运行暴露了一个很典型的 agent 失败：它做了大量研究，然后试图把报告写到磁盘上的 markdown 文件里。由于它运行在 notebook 环境中，写文件权限失败，输出也随之变糟。这个失败很有教学价值，因为最终输出看起来像“报告生成失败”，trace 却显示研究步骤基本完成，真正问

⁶视频画面时间区间：00:35:38–00:38:12。

题是 agent 把 “write a report” 理解成 “在磁盘上创建报告文件”。prompt 需要约束输出位置，例如 “把报告直接返回给用户”。

FIGURE_SLOT: apple-write-to-disk-failure

建议插入 Apple trace 中写磁盘失败的 span 细节，重点显示重复 write 失败、notebook 权限问题与最终输出缺陷。

图 7: Apple 运行中的写磁盘失败：trace 揭示了最终输出背后的 root cause。⁷

Nvidia 运行展示了另一类问题：报告很完整，也正确写回了 agent 输出，可是缺少 investment actionability。它描述了竞争格局、市场地位和前景，却没有明确告诉用户如何行动。Rivian 例子则提醒读者检查证据：报告里出现看似很精确的交付数量，如果数据源本身难以核验，就应该回到 trace 里看 Web Search 是否真的提供了可信来源。这里要格外谨慎，字幕中的 Rivian 描述可能带有 ASR 或事实误差，最终文档只把它当成 “可疑精确数字需要来源核验” 的例子。

1.8 失败分类：severity 与 frequency

读完多条 trace 后，团队要把单个失败变成分类。视频里给出的粗略 root-cause categories 包括：mostly looks good、possible hallucination、reasoning gaps、unverifiable data、missing recommendations。分类的目的在于决定接下来先修什么，表格只是承载形式。

Failure category	trace reading 线索	可能修复方向
mostly looks good	输出和中间步骤大体合理，缺陷很小或暂不明显	保留为通过样本或低优先级复查样本
possible hallucination	出现过于具体、来源不清、与搜索证据脱节的数字或结论	增加 faithfulness/source-grounding eval，要求引用证据
reasoning gaps	搜索到了资料，结论跳跃或没有连接证据与建议	强化报告结构 prompt，加入推理检查或 LLM-as-a-judge
unverifiable data	trace 中找不到足够可信来源支撑关键事实	增加来源质量要求、工具约束与人工复核路径
missing recommendations	报告描述充分，却没有 buy/sell/hold 或下一步建议	设计 actionability rubric 与对应 custom eval

表 1: 从 trace reading 到失败分类的示例表。

FIGURE_SLOT: root-cause-frequency-table

建议插入或重绘 root-cause frequency 表/图，包含 mostly looks good、possible hallucination、reasoning gaps、unverifiable data、missing recommendations。

图 8: trace 分类后的 root-cause frequency 汇总。⁸

只看 frequency 会误导优先级。一个轻微幻觉频繁出现，当然值得修；一次严重越界也可能更

⁷视频画面时间区间：00:43:13–00:45:44。

⁸视频画面时间区间：00:47:27–00:48:35。

危险。讲者给出的朴素排序规则是把严重程度 **severity** 与出现频率 **frequency** 相乘：

$$P = S \times F$$

其中 P 是修复优先级， S 是 **severity**，表示失败对用户、业务、合规和信任的伤害程度； F 是 **frequency**，表示该失败在样本或生产流量中出现的频率。这个公式并非严格风险模型；更准确地说，它是一个工程排序启发式：高伤害且高频的失败优先修。

风险排序的隐藏假设

$P = S \times F$ 默认团队能相对稳定地估计严重程度，并且样本频率接近真实流量。若数据集偏向热门科技股，**frequency** 会低估冷门公司失败；若只看 demo 环境，**severity** 会低估真实用户把金融报告用于决策时的风险。工程团队需要把生产分布、监管边界和人工复核成本一起纳入判断。

1.9 Swiss cheese model：分层质量防线

Swiss cheese model（瑞士奶酪模型）来自安全工程：每一层防线都像一片奶酪，有洞；事故发生在多个洞恰好对齐时。放到 agent eval 里，code eval、LLM-as-a-judge、人类 review、生产监控、数据集实验都是不同防线。任何单层都会漏：code eval 擅长确定性条件，难以理解语义；LLM judge 能抓 reasoning gaps，可能漏掉细微幻觉或受到偏见影响；human review 最可靠的一面是常识和领域判断，可是无法覆盖每一条 trace。

FIGURE_SLOT: swiss-cheese-model-eval-layers

建议插入讲者引用的 Swiss cheese model 图，或重绘为 code eval、LLM judge、human review、monitoring 多层防线。若使用视频中的 Anthropic 博客图，caption 需说明这是讲者引用的安全工程模型。

图 9: Swiss cheese model：多层 eval 防线降低失败穿透概率。⁹

一个简单的概率直觉

若第 k 层防线漏掉某类失败的概率是 q_k ，并且各层失败近似独立，那么全部漏掉的概率可以直觉化为：

$$Q = \prod_{k=1}^m q_k$$

其中 m 是防线层数， Q 是失败穿透所有层的概率。真实系统里各层常常相关，例如同一个模糊需求会同时误导 prompt、rubric 和人工标注指南，所以这个公式只能帮助理解“分层会降低风险”，不能替代真实 meta-evaluation。

这一模型给本节一个清晰收束：trace reading 是设计防线的原始证据，最终防线需要由多层 eval、review 和 monitoring 共同承担。先读 trace，找到失败类别；再把类别转成 code eval、faithfulness、actionability judge、人类标注和后续 experiment。这样构建出来的 eval suite 才具有工程系统的形态，避免停留在一次性 notebook 演示。

⁹视频画面时间区间：00:48:45–00:49:45。

1.10 本章小结

本节完成了从 instrumentation 到 trace reading 的第一条主线。eval 结果由 **score**、**label** 和 **explanation** 组成，其中 **explanation** 让失败变得可修复；Phoenix 通过 OpenTelemetry、OTel 和 OpenInference 捕获 agent 的 trace/span，并在 UI 中展示 annotation、attribute、token count、cost、latency、输入和输出；financial-analysis demo agent 使用 Claude Agent SDK、Claude Haiku 与 Web Search 构造了一个足够简单、足够会出错的课堂系统；13 条 trace 让团队开始看到 Apple 写磁盘失败、Nvidia 缺少 actionability、Rivian 数字需核验等不同问题；最后，severity 与 frequency 帮助决定修复顺序，Swiss cheese model 说明多层 eval 防线为何比单一指标更可靠。

读者应带走的实践顺序很朴素：先让系统可观测，再读真实 trace，然后写下“好输出”的具体定义，接着把失败分类转成 eval。后续章节会把这个证据层继续向前推进：先写确定性的 code eval，再引入 built-in LLM judges，最后把本节发现的 missing recommendations 转成专门的 actionability rubric。